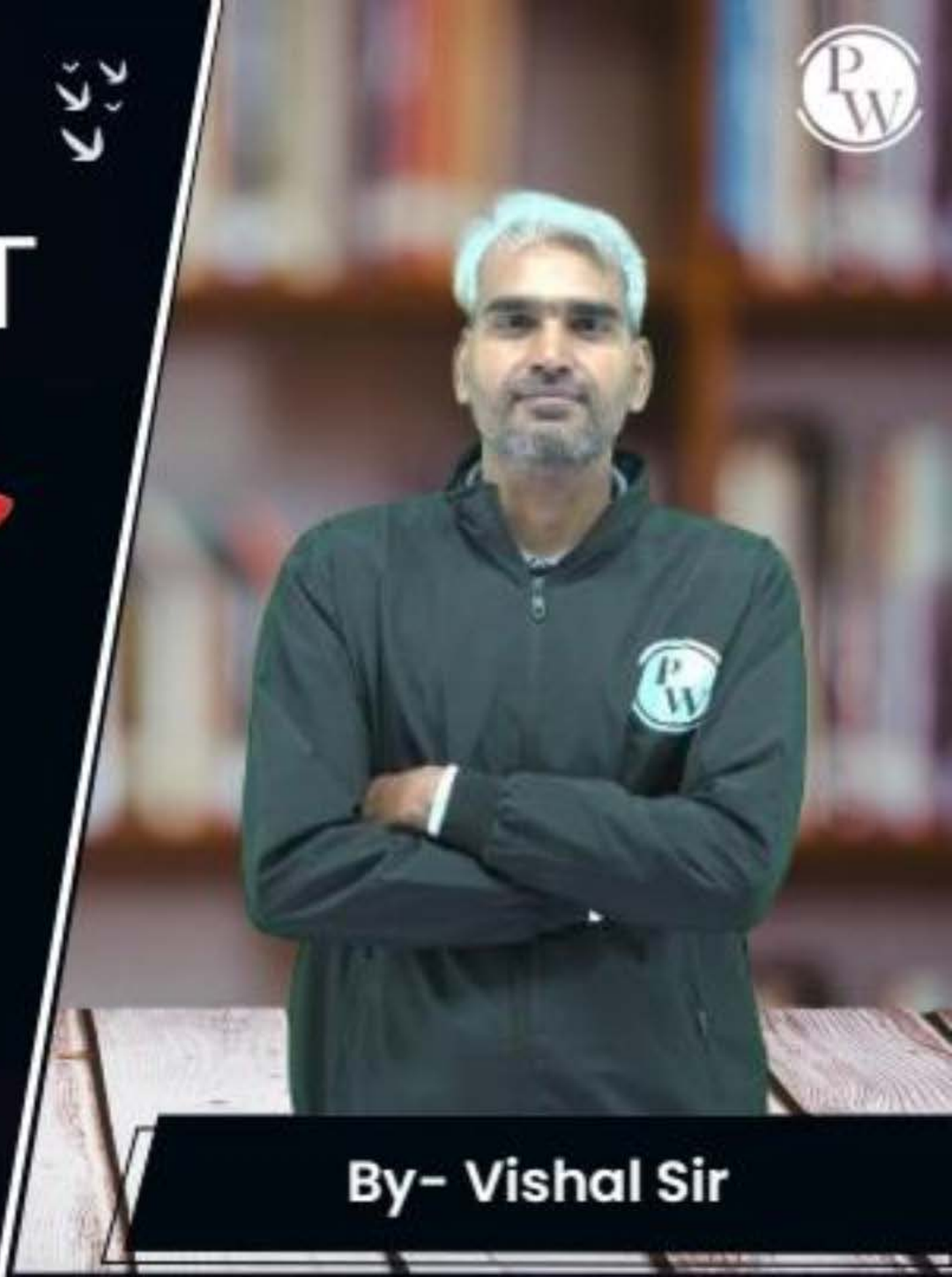
Computer Science & IT



Database Management System

Relational Model & Normal Forms

Lecture No. 02



By- Vishal Sir

Recap of Previous Lecture



- Topic: Syllabus
- Topic: Introduction to DBMS

Slide

Topics to be Covered



- Topic: Limitations of file system
- Topic: Introduction Relational Database Model
- Topic: Functional dependency
- Topic: Types of functional dependency

Slide



Topic : Introduction to DBMS

Database is organized collection of related information

Data → {raw facts} → May not provide any information

{10, 20, 35, 60}

Information → {Data with meaning}

10 - No. of students in Vijay batch
20 - No. of students in Parakram batch
35 - No. of students in Shreshtha batch
60 - No. of students in Super60 batch

Database

DBMS : Database management system is a software used to manage and access the database efficiently.

Information can be organized in multiple ways, so different database models are used

eg: ER model
Network model
Hierarchical model
Relational model
object oriented model
etc.

In this course our main focus will be on relational model

In relational model information is organized in the form of table

- → If we store the information gathered in the previous slide in the form of table.

i.e.:

Batch_name	No-of-students
Vijay	10
Parakram	20
Shreshtha	35
Super60	60

And it satisfies the constraints specified by relational model, then it can be called a relation

- "File system" can be used to manage and access the database, but file system fails to do the task efficiently if database is too large.
- Database files are stored in the non-volatile memory i.e; secondary memory.
- Unit of transfer between secondary memory and main memory is "one disk block".

✓ IO Cost:- IO cost of a "record access" can be defined as number of secondary memory disk blocks that needs to be transferred from secondary memory to main memory in order to access that record from the database

{ it is stored in the blocks of the disk }

Student.txt file

"other information related to Student"

	Sid	Sname	- - - - - -
B_1	S_1 S_{100}		
B_2	S_{150} S_4		
B_3	S_{227} S_{15}		
B_4	S_{225} S_3		
	⋮	⋮	⋮
B_N	S_2 S_{11}		

Let 'N' blocks of the disk are required to store the file Student.txt

(Too large)

* Retrieve record of the student with Sid = S_{240}

{ Manually search for record }

↓

It will be time Consuming

{ Write a program to search for Sid = S_{240} }

↓

To write a program we need to know the Physical storage details of the file.

{ it may be a security issue }

{ it is stored in the blocks of the disk }

Student.txt file — "other information related to Student"

	Sid	Sname	- - - - - -
B_1	S_1 … S_{100}		
B_2	S_{150} … S_4		
B_3	S_{227} … S_{15}		
B_4	S_{225} … S_3		
	⋮	⋮	⋮
B_N (Too large)	S_2 … S_{11}		

Let 'N' blocks of the disk are required to store the file Student.txt

- Retrieve record of the student with Sid = S_{240}

Even if we have written a program to compare the existing Sids with "S_{240}", then we will have to transfer the blocks of file Student.txt from secondary memory to main memory (one by one) until the block containing the record with Sid = S_{240} is found.

In worst case we may have to transfer all the blocks of the file from secondary memory to main memory & it will be time consuming.

i.e; with file system IO cost may be too high.

{ it is stored in the blocks of the disk }

Student.txt file

"other information related to Student"

	Sid	Sname	- - - - - -
B_1	S_1 S_{100}		
B_2	S_{150} S_4		
B_3	S_{227} S_{15}		
B_4	S_{225} S_3		
	⋮	⋮	⋮
B_N	S_2 S_{11}		

Let 'N' blocks of the disk are required to store the file Student.txt

(Too large)

Consider two users U_1 & U_2

U_1: wants to update (write) record of Student with Sid = S_{15}

U_2: wants to update (write) record of Student with Sid = S_3

{ it is stored in the blocks of the disk }

Student.txt file — "other information related to Student"

	Sid	Sname	- - - - - -
B_1	S_1 … S_{100}		
B_2	S_{150} … S_4		
B_3	S_{227} … S_{15}		
B_4	S_{225} … S_3		
	⋮	⋮	⋮
B_N	S_2 … S_{11}		

Let 'N' blocks of the disk are required to store the file Student.txt (Too large)

Consider two users U_1 & U_2 operation using file system

U_1	U_2
lock(Student.txt)	
Update(S_{15})	
	lock(Student.txt) ← This request will be denied ∴ U_2 will not be able to perform its operation

let granted

time

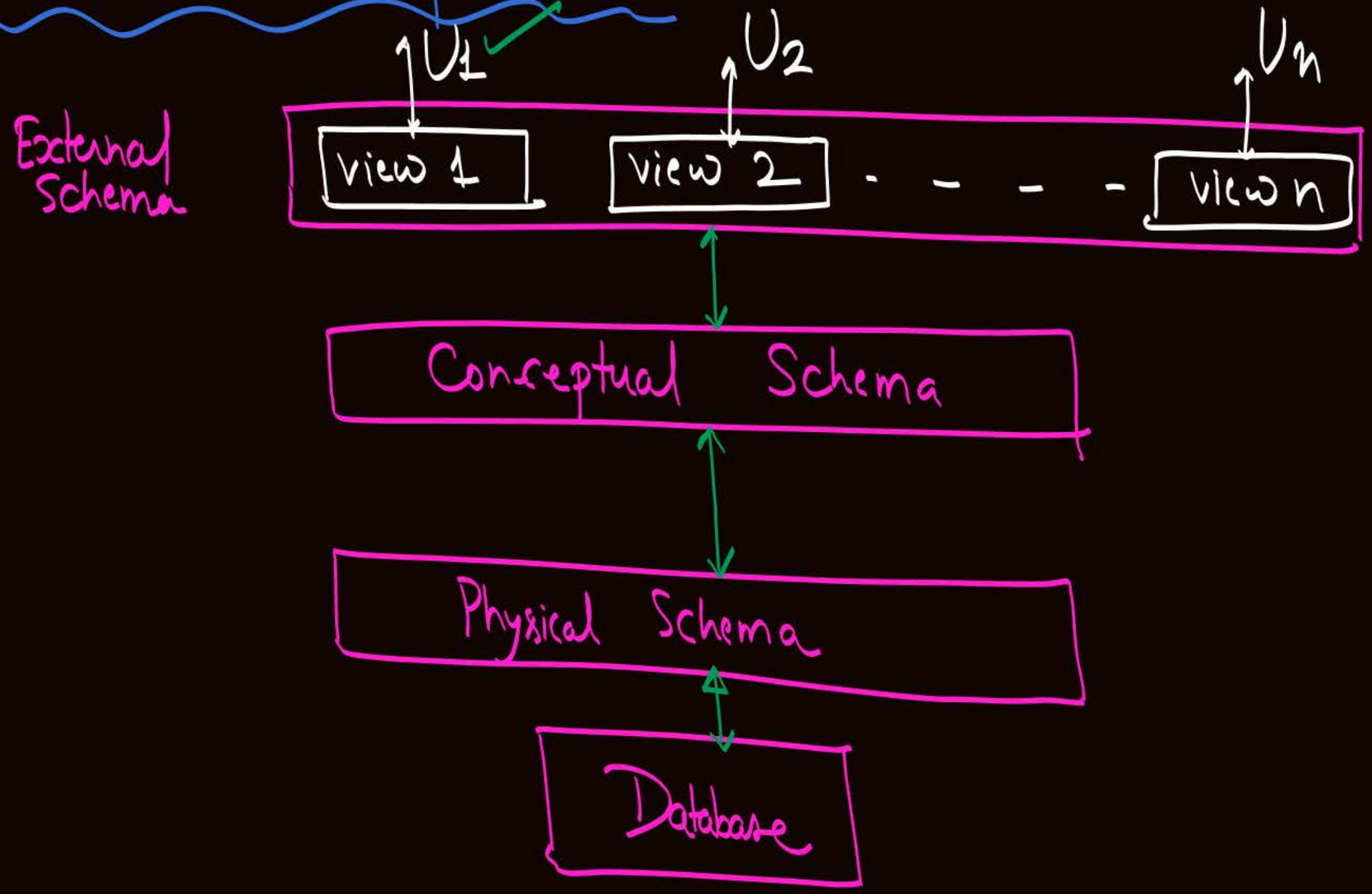
∴ With file system Concurrency is low.

User U_1 & U_2 are not allowed to update the records simultaneous, even when records are with respect to different Students.

File System vs DBMS :-

File System	vs	DBMS
① If we want to access the record from the file using a program, then in order to write the program we need to know physical details of the file.		① DBMS uses 3-tier architecture. 3-tier architecture of DBMS provides, – (i) Data abstraction : {It will allow us to hide the physical details from external user} – (ii) Data independence : {Changes performed at lower level will not affect higher levels}
② More IO Cost		② less IO Cost └ In DBMS Concept of index file is used to reduce IO Cost.
③ Low Concurrency : {In file system locking is at file level} ∴ Concurrency is low		③ High Concurrency level :- In DBMS, locking will be at record level ∴ High Concurrency level

⇒ 3-tier Architecture of DBMS:-

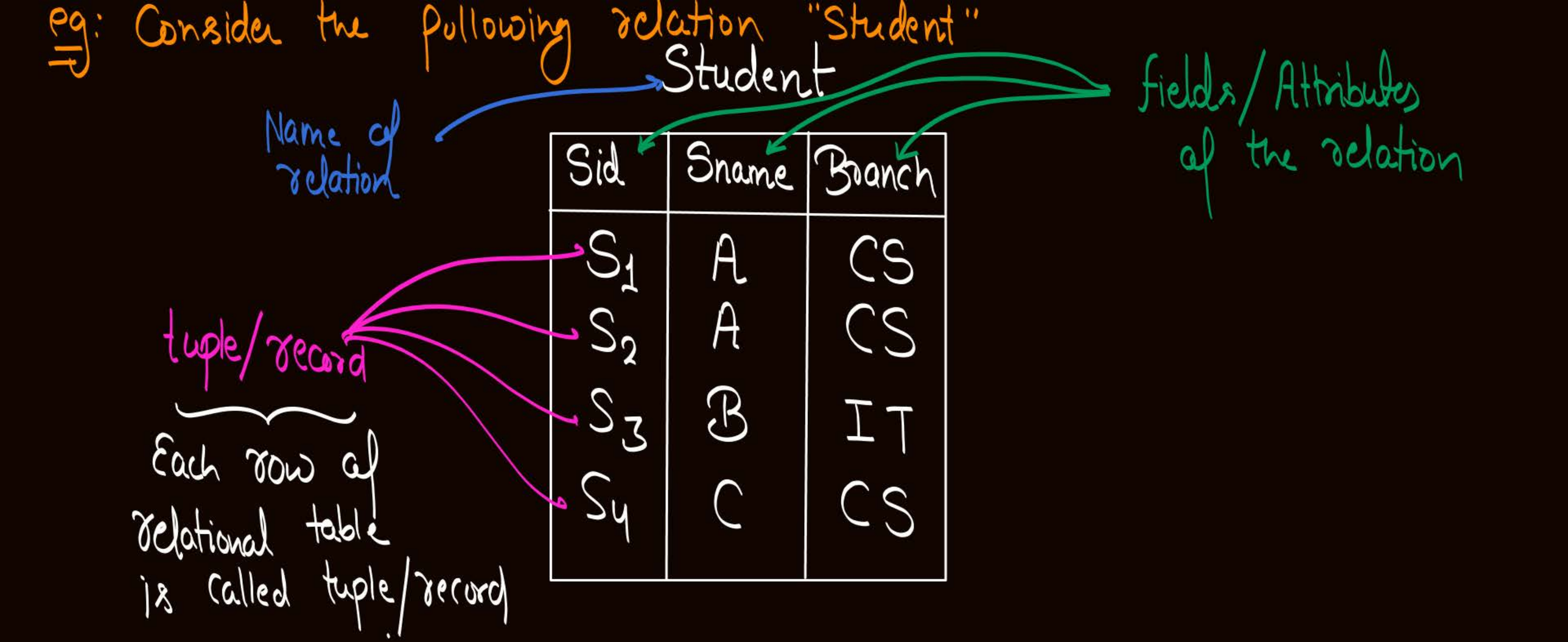




Topic : Relational database

- In relational database information will be organized in the form of table {i.e., Collection of rows and Columns}
- Each table of relational database is called a relation
- Dr. Codd defined 13 rules {from "rule 0" to "rule 12"} for a table to be called a relation.

eg: Consider the following relation "Student"



* **Degree / Arity :** The number of attributes (Columns) in the relation is defined as degree/arity of that relation

* **Cardinality :-** Number of tuples/record in a relational table define the cardinality of that relation

* **Relational Schema :-** Relational schema provides the abstract detail of the relation

eg: Name_of_Relation(1st attribute in Relⁿ , 2nd attribute in Relⁿ , 3rd attribute in Relⁿ , - - - -)

order is important

* **Relational instance :-** If the data/record are present in the relation, then set of all those tuple will define the relational instance of that time

{ relational instance may change with time because of insert, delete or update oph }

eg: Consider the following relation "Student"

Student

Name of relation

fields / Attributes of the relation

Sid	Sname	Branch
S_1	A	CS
S_2	A	CS
S_3	B	IT
S_4	C	CS

* Degree/Arity = 3
* Cardinality = 4
* Relational Schema w.r.t. Student Relation is
  Student (Sid, Sname, Branch)
* Relational instance (for current instance) = { (S_1, A, CS), (S_2, A, CS), (S_3, B, IT), (S_4, C, CS) }



Topic : Functional dependency (FD)

Functional dependencies are used to define the relationship among the attributes of relation

Functional dependencies are denoted by format,

$$X \longrightarrow Y$$

Where X and Y are some set of attributes from the relation



Topic : Functional dependency (FD)

In functional dependency $X \longrightarrow Y$,

'X' is called determinant,

& 'Y' is called dependent.



Topic : Functional dependency (FD)

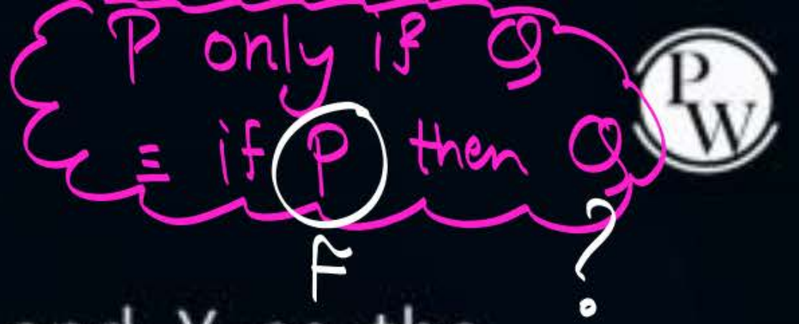
- Functional dependency defines the relationship between two sets of attributes in a relational table.

- It states that the values of dependent attribute set can be completely determined by the values of determinant attributes set. In other words, if a determinant attribute's value is known, the value of the dependent attribute can be deduced.

i.e. in $X \longrightarrow Y$
if we know the value of X
then we can deduce the value of Y from the relation



Topic : Functional dependency (FD)



- Let R be the relational schema with X and Y as the attribute sets over relation R.

(Functional dependency X→Y exists in R) only if
(For all pair of tuples t1,t2∈R
If t1.X = t2.X then t1.Y = t2.Y)

* if Functional dependency X→Y exists in R
then
(for ∀ $t_1, t_2 \in R$
if $t_1.X = t_2.X$ then $t_1.Y = t_2.Y$)



Topic : Functional dependency (FD)

if $P \rightarrow Q$
then $Q \rightarrow P$

* If functional dependency $X \rightarrow Y$ exists in relation R, then

Whenever values w.r.t. attribute set X are same in any pair of tuples, then values of attribute set Y will also be same in those tuples

It is the necessary condition for $X \rightarrow Y$ to exist in the relation, but not sufficient

But if values w.r.t attribute set Y are same in any pair of tuples, then values w.r.t. attribute set X need not be same in those tuples.



2 mins Summary

Topic Limitations of file system

Topic Introduction Relational Database Model

Slide

THANK - YOU